

MACHINE LEARNING PROJECT REPORT

Prediction of the grades of students

December 2025

MMN1, A4 ESILV

Daphne BARAY, Sofiane BEAUMONT, Max BAUMBERGER

Contents

1	Introduction	2
2	Data exploration and analysis	3
2.1	Feature description	3
2.2	Data Quality Assessment	4
2.2.1	Data Types and Consistency	4
2.2.2	Missing Values Analysis	4
2.2.3	Duplicate Check	5
2.3	Exploratory Data Analysis	5
2.3.1	Distribution of the Target Variable	5
2.3.2	Key Correlations (Numerical Variables)	6
2.3.3	Categorical Insights	6
3	Preprocessing	7
3.1	Feature Selection	7
3.2	Encoding Categorical Variables	7
3.3	Handling Missing Values	8
3.4	Feature Scaling	8
3.5	Train/Test Split	8
4	Machine Learning Models and Optimization	8
4.1	Baseline Model: Decision Tree	8
4.2	Ensemble Method: Bagging	9
4.3	Advanced Model: XGBoost	9
4.4	Hyperparameter Tuning Strategy	9
4.4.1	Grid Search for Decision Tree	9
4.4.2	Randomized Search for XGBoost	9
4.4.3	Optimization Metric	10
5	Analysis of Results	10
5.0.1	Error Analysis by Actual Score	11
6	Conclusion	11

1 Introduction

Note : Ce document contient des liens hypertextes menant vers le dataset [Kaggle](#), le répertoire [GitHub](#) et le notebook [Google Colab](#).

In the modern educational landscape, understanding the factors that drive academic success is a key issue for schools and families. A student's performance is rarely the result of a single cause; instead, it comes from a complex mix of study habits, social support, and personal circumstances.

The main goal of this project is to create a predictive model to estimate a student's Exam_Score based on these different variables. By applying Machine Learning techniques, we aim to move beyond simple observation and build a tool capable of anticipating future results. Since we are predicting a continuous numerical value, this is treated as a regression problem.

To achieve this, we used the StudentPerformanceFactors.csv dataset, that we found on [Kaggle](#). This dataset contains a variety of information about the ~6600 students who participated to have their data in this dataset, including their academic behaviors and social environment. Analyzing this data allows us to identify the most influential factors and propose a model that is both accurate and reliable.

This report follows the standard machine learning workflow. First, we will present a descriptive analysis and the steps taken to clean and prepare the data. Then, we will detail the modeling phase, starting with a basic Decision Tree and moving to a more advanced XGBoost model. Finally, we will compare the results to determine which approach offers the best performance.

We started by creating a [GitHub](#) repository and a [Google Colab](#) notebook file to keep track and work on the project together.

2 Data exploration and analysis

2.1 Feature description

The dataset consists of 20 variables in total, which can be categorized into numerical (quantitative) and categorical (qualitative) types.

Numerical Variables

These variables represent measurable quantities related to the student's performance and habits.

- **Hours_Studied (Integer):** The number of hours the student spends studying per week. This is a key indicator of academic effort.
- **Attendance (Integer):** The percentage of classes attended by the student. Higher attendance is often linked to better understanding of the material.
- **Previous_Scores (Integer):** The scores obtained by the student in previous exams. This helps assess the student's historical academic level.
- **Sleep_Hours (Integer):** The average number of hours the student sleeps per night. This reflects the student's physical well-being and rest.
- **Tutoring_Sessions (Integer):** The number of tutoring sessions the student has attended per month. This indicates the level of extra academic support received.
- **Physical_Activity (Integer):** The average number of hours of physical activity per week. This variable measures the student's lifestyle balance.
- **Exam_Score (Integer):** Target Variable. The final score obtained by the student in the exam (0-100). This is the value we aim to predict.

Categorical Variables

These variables describe the student's environment, background, and personal characteristics.

- **Parental_Involvement (Object/String):** Level of parental involvement in the student's education ('Low', 'Medium', 'High').
- **Access_to_Resources (Object/String):** Availability of educational resources like internet, books, etc. ('Low', 'Medium', 'High').
- **Extracurricular_Activities (Object/String):** Participation in non-academic activities ('Yes', 'No').
- **Motivation_Level (Object/String):** The student's self-assessed level of motivation ('Low', 'Medium', 'High').
- **Internet_Access (Object/String):** Indicates if the student has internet access at home ('Yes', 'No').
- **Family_Income (Object/String):** The income level of the student's family ('Low', 'Medium', 'High').

- **Teacher_Quality (Object/String):** The perceived quality of the teachers ('Low', 'Medium', 'High'). Contains missing values.
- **School_Type (Object/String):** The type of school attended ('Public', 'Private').
- **Peer_Influence (Object/String):** The influence of friends and peers on the student's academic life ('Positive', 'Negative', 'Neutral').
- **Learning_Disabilities (Object/String):** Indicates if the student has been diagnosed with a learning disability ('Yes', 'No').
- **Parental_Education_Level (Object/String):** The highest education level attained by the parents ('High School', 'College', 'Postgraduate'). Contains missing values.
- **Distance_from_Home (Object/String):** The distance between the student's home and school ('Near', 'Moderate', 'Far'). Contains missing values.
- **Gender (Object/String):** The gender of the student ('Male', 'Female').

2.2 Data Quality Assessment

Before proceeding with the analysis, we performed a rigorous check of the dataset's quality to identify any structural errors or missing information.

2.2.1 Data Types and Consistency

We verified that all variables were assigned the correct data types.

- Numerical variables (Hours_Studied, Exam_Score..) were correctly loaded as integers.
- Categorical variables (Parental_Involvement, School_Type..) were loaded as objects (strings). No immediate type conversion was required at this stage, although categorical variables will require encoding during the preprocessing phase.

2.2.2 Missing Values Analysis

We analyzed the dataset for null values and identified that 3 variables contained missing data :

- Teacher_Quality
- Parental_Education_Level
- Distance_from_Home

However, the proportion of missing data was very low (less than 1.36% of the total observations). This limited amount of missing information suggests that the dataset is generally complete and reliable. Because the percentage is small, we chose to handle these values through imputation (replacing them) rather than deleting the rows, to preserve as much data as possible for training.

Teacher_Quality	1.180566
School_Type	0.000000
Peer_Influence	0.000000
Physical_Activity	0.000000
Learning_Disabilities	0.000000
Parental_Education_Level	1.362192
Distance_from_Home	1.014076
Gender	0.000000
Exam_Score	0.000000
dtype:	float64

Figure 1: Percentage of missing values in categorial variables

2.2.3 Duplicate Check

We scanned the dataset for duplicate rows. The analysis confirmed that no duplicates were present (0 duplicates found), ensuring that every observation represents a unique student.

2.3 Exploratory Data Analysis

Once the data was validated, we conducted a visual analysis to understand the distributions and relationships between variables.

2.3.1 Distribution of the Target Variable

We examined the distribution of the Exam_Score. The histogram shows that the scores follow a normal distribution (a bell-shaped curve), centered around a mean of approximately 67. This indicates that most students perform near the average, with fewer students achieving very low or very high scores. This standard distribution confirms that the target variable is well-behaved and suitable for regression models.

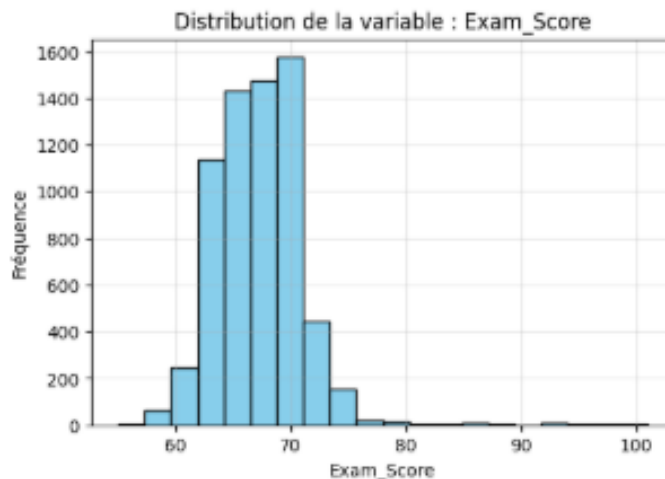


Figure 2: Distribution of the Exam_Score variable

2.3.2 Key Correlations (Numerical Variables)

To identify the most predictive factors, we calculated the correlation between numerical variables and the Exam_Score.

- **Hours_Studied** and **Attendance** showed the strongest positive correlations. This confirms the intuitive expectation that more study time and class presence lead to higher grades.
- **Sleep_Hours** and **Previous_Scores** also displayed positive relationships, though slightly weaker.

The correlation matrix helped us confirm that there were variables too correlated with each other that could confuse the model.

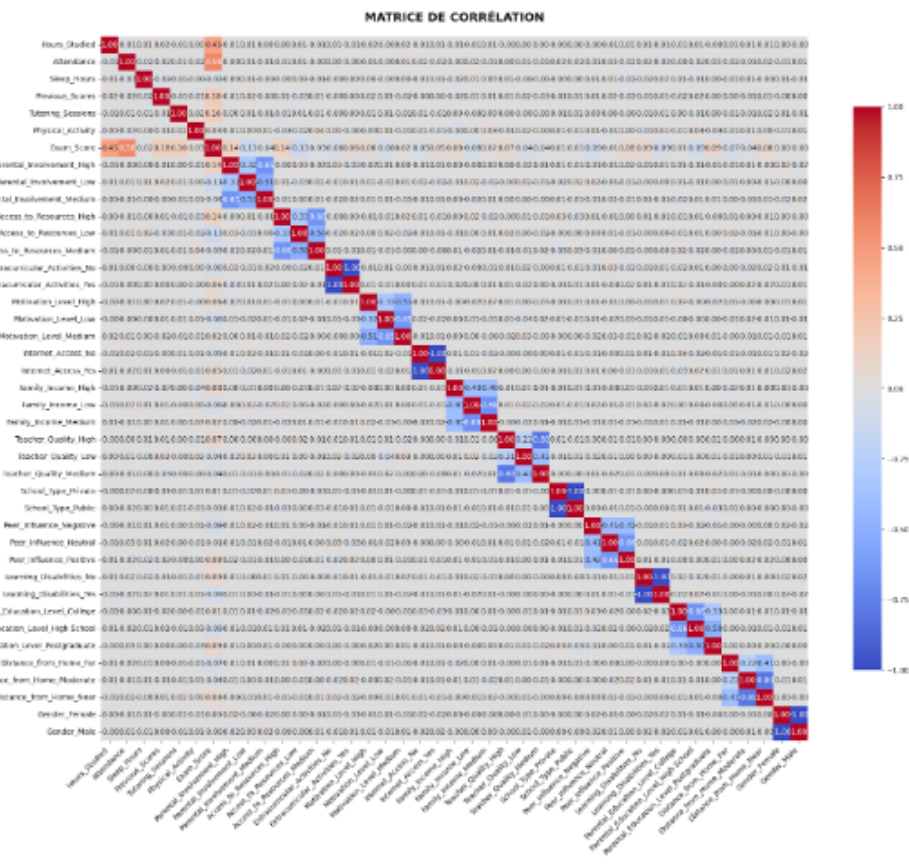


Figure 3: Correlation Matrix

This correlation matrix was obtained after preprocessing, and especially after encoding the categorical variables (3.2).

2.3.3 Categorical Insights

We analyzed how categorical variables impact performance using box plots and bar charts.

- **Parental_Involvement:** Students with "High" parental involvement tended to have higher median exam scores compared to those with "Low" involvement.
- **Access_to_Resources:** Similarly, better access to educational resources was associated with improved performance.

- **Motivation_Level:** Contrary to what we could think, there is no clear trend where higher self-reported motivation correlates with better results.

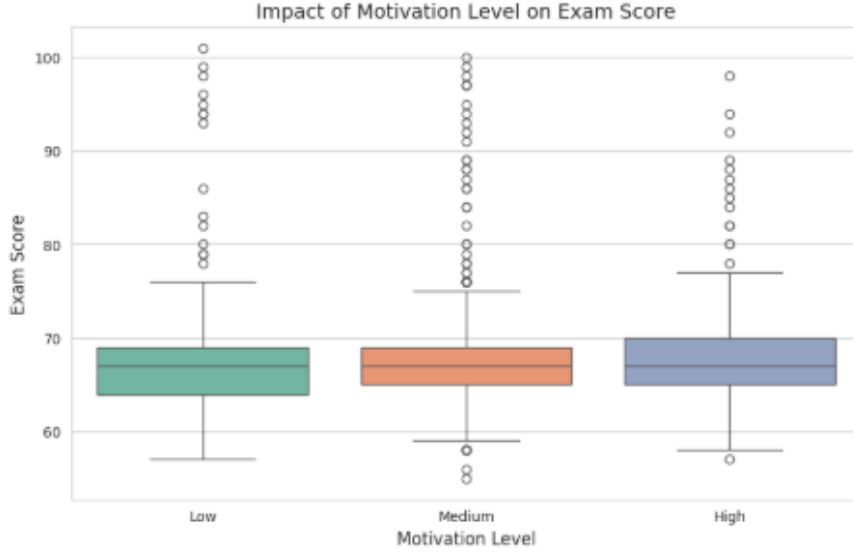


Figure 4: Boxplots of the exam scores in function of the different motivation levels

This exploration highlights that student performance is driven by a combination of personal habits (study, sleep) and environmental support (parents, resources). These insights guided our feature selection for the machine learning models.

3 Preprocessing

Before feeding the data into machine learning models, we applied a series of transformations to ensure the dataset was clean, numerical, and optimized for learning.

3.1 Feature Selection

To improve model efficiency and reduce noise, we performed feature selection based on correlation. We calculated the correlation of each new variable with the target (Exam_Score). We decided to keep only the variables that showed a correlation coefficient of at least 0.1 (absolute value). This ensures the model focuses only on the most relevant factors, reducing complexity and the risk of overfitting. As a result, variables with very weak relationships to the target were removed.

3.2 Encoding Categorical Variables

Machine learning algorithms require numerical input. Since our dataset contained many categorical variables (like Parental_Involvement or School_Type), we transformed them using One-Hot Encoding. This process converts each category into a new binary column (0 or 1). For example, a "Gender" column is split into "Gender_Male" and "Gender_Female". This resulted in an expanded dataset where all features were numeric.

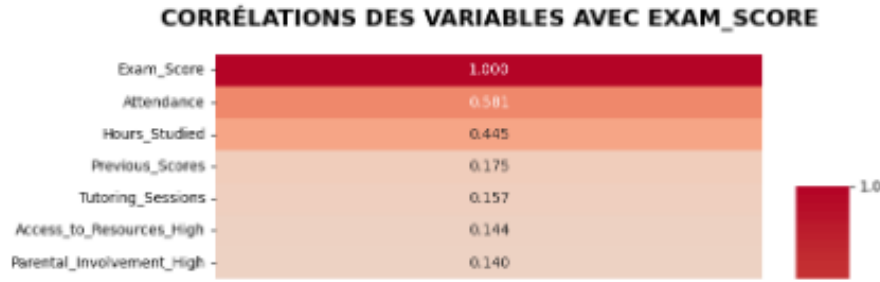


Figure 5: Correlation Variables with Exam_Score

3.3 Handling Missing Values

As identified in the data quality assessment, three categorical variables (Teacher_Quality, Parental_Education_Level, Distance_from_Home) contained a small number of missing values. To address this, we used Mode Imputation. We replaced the missing entries with the most frequent category of each respective column. This method allows us to retain all observations without introducing artificial noise.

3.4 Feature Scaling

After filtering the variables, we applied Standard Scaling (StandardScaler) to the entire feature set (X). This transformation standardizes the data so that each variable has a mean of 0 and a standard deviation of 1. Scaling is crucial because it prevents variables with large ranges (like Hours_Studied) from dominating variables with smaller ranges during the training process.

3.5 Train/Test Split

Finally, we split the processed dataset into two distinct subset: a training set containing 70% of our data, used to train the models, and a test set with the rest (30%) that's kept separate and used only to evaluate the final performance. We used a fixed random_state=42 to ensure the split is reproducible.

4 Machine Learning Models and Optimization

We implemented a progressive modeling strategy by testing three distinct approaches to maximize predictive accuracy, starting with a simple Decision Tree and advancing to an ensemble Bagging model and finally an optimized XGBoost algorithm.

4.1 Baseline Model: Decision Tree

We began our analysis with a Decision Tree Regressor as our baseline model because it is intuitive and easy to interpret. The initial unoptimized tree showed signs of overfitting, meaning it learned the training data too specifically and struggled to generalize to new students. To address this, we used GridSearchCV to systematically test various combinations of hyperparameters such as the maximum depth of the tree and the minimum

samples required to split a node. The best configuration found by the search significantly reduced the variance of the model, providing a much more stable and reliable baseline for further comparison.

4.2 Ensemble Method: Bagging

Building on the optimized Decision Tree, we implemented a Bagging Regressor to further improve performance. This method creates multiple versions of the same model (in this case, 100 optimized trees) trained on random subsets of the data and then averages their predictions. This approach proved effective because averaging the results of many trees generally reduces the risk of errors and makes the model more robust to outliers in the student data compared to a single tree.

4.3 Advanced Model: XGBoost

Finally, we tested XGBoost (Extreme Gradient Boosting), a state-of-the-art algorithm known for its high performance in regression tasks. Unlike Bagging, which builds trees in parallel, Boosting builds them sequentially so that each new tree attempts to correct the errors of the previous one. Given the complexity of this algorithm, we used RandomizedSearchCV to efficiently find the best hyperparameters by testing 40 different combinations of settings like learning rate, number of estimators, and tree depth. The search identified an optimal configuration that provided the highest accuracy of all the models we evaluated.

4.4 Hyperparameter Tuning Strategy

As said, to ensure our models performed at their best potential, we did not rely on default settings. Instead, we implemented a rigorous optimization phase to identify the most effective hyperparameters for each algorithm.

4.4.1 Grid Search for Decision Tree

For the Decision Tree model, we employed GridSearchCV (Grid Search with Cross-Validation). Since the number of parameters to tune was relatively small, we could afford to test every possible combination in our defined search space. We systematically varied the `max_depth` to control complexity, along with `min_samples_split` and `min_samples_leaf` to prevent the tree from growing too specific to the training data. We used 5-fold Cross-Validation, meaning the model was trained and validated 5 times on different subsets of data for each combination, ensuring the results were reliable and not due to chance.

4.4.2 Randomized Search for XGBoost

For the XGBoost model, the search space was significantly larger and more complex. Testing every combination would have been computationally too expensive. Therefore, we adopted RandomizedSearchCV. Instead of checking every point, this method randomly sampled 40 different configurations from a specified distribution of hyperparameters. We adjusted boosting parameters, such as `learning_rate` (how fast the model learns), `n_estimators` (number of trees), and `max_depth`. We used 3-fold Cross-Validation to

evaluate each of the 40 iterations. This approach allowed us to explore a wide range of possibilities efficiently and find a high-performing configuration without excessive calculation time.

4.4.3 Optimization Metric

For both strategies, we optimized the models to minimize the MSE. This ensures that the selected models are those that make the smallest average errors in their predictions.

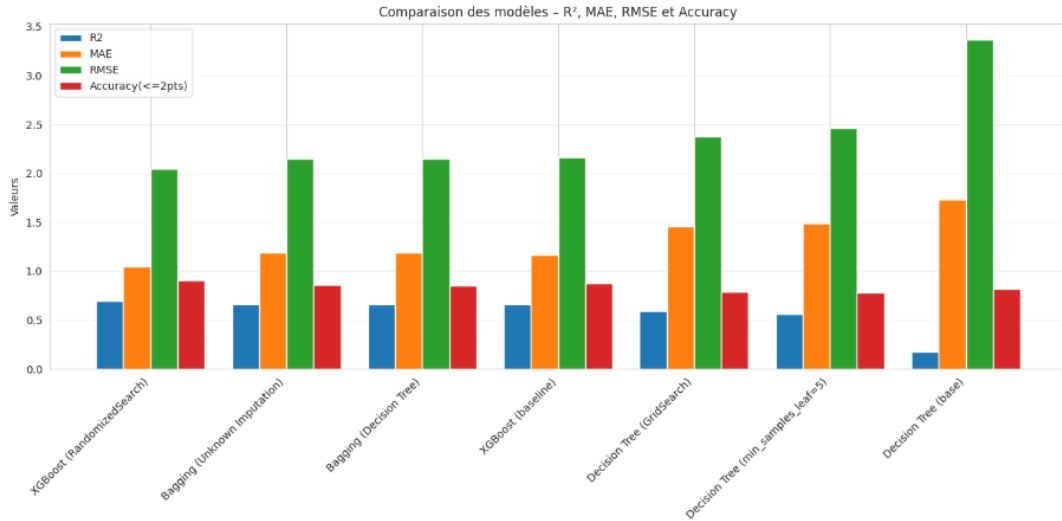


Figure 6: Comparison of the models we tested

5 Analysis of Results

After training and optimizing our three candidate models, we compiled their performance metrics to identify the most effective solution. The results reveal a clear hierarchy in model performance:

- Decision Tree (Baseline) showed the lowest predictive power with an R^2 of roughly 0.59 (optimized). While interpretable, it struggled to capture the complex, non-linear relationships between student habits and exam scores.
- Bagging Regressor significantly improved upon the single tree, reaching an R^2 of 0.666. By averaging 100 trees, it successfully reduced variance and smoothed out errors.
- XGBoost emerged as the best performing model with an R^2 of 0.70 and the lowest Root Mean Squared Error ($RMSE \approx 2.03$).

The superiority of XGBoost can be attributed to its boosting mechanism. Unlike Bagging, which builds trees independently, XGBoost learns sequentially. It specifically targeted the "hard-to-predict" students where previous trees had failed, iteratively correcting errors. This allowed it to fine-tune its predictions more effectively than the other models.

The final MAE of 1.04 for the XGBoost model is particularly encouraging. In practical terms, this means that on average, our model's prediction is only 1 point away from a

student's actual exam score (on a scale of 0-100). For an educational tool intended to flag students at risk or estimate performance, this level of precision is highly reliable.

5.0.1 Error Analysis by Actual Score

To critically assess the model's reliability across the entire score range, we conducted a detailed error analysis, defining a prediction as "correct" if the absolute error was ≤ 2 points. The analysis showed that precision is directly proportional to score frequency: Best Performance: Accuracy is highest 95 Highest Errors: The largest errors are systematically found on rare, extreme scores (both high and low). This confirms the model tends to generalize where training data is rare (scores 98, 87, which had 0 This indicates the model is highly robust for the average student but less reliable for predicting highly exceptional or extremely low performance.

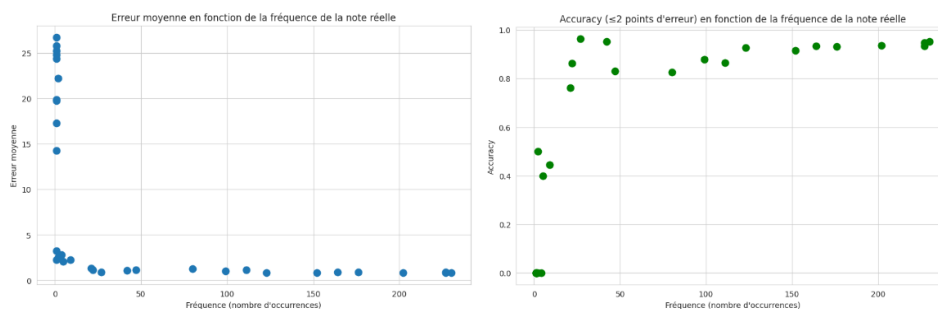


Figure 7: Average error and Accuracy as a function of frequency

6 Conclusion

This project aimed to predict student exam scores using machine learning techniques based on various demographic, social, and academic factors. By analyzing the Student-PerformanceFactors.csv dataset, we successfully built and optimized several predictive models to address this regression problem.

Our exploratory analysis and modeling confirmed that student success is largely driven by measurable behaviors. Academic habits such as attendance and weekly study hours emerged as the strongest predictors of performance, whereas background factors like teacher quality or distance from home had a negligible impact. In terms of modeling, we observed a clear progression in accuracy. The baseline Decision Tree was too simple to capture the full complexity of the data, resulting in an R^2 of approximately 0.59. The introduction of the Bagging Regressor improved robustness by averaging predictions, raising the score to 0.66. Ultimately, the XGBoost algorithm proved to be the most effective solution, achieving an R^2 of roughly 0.70 and a low Mean Absolute Error of about 1 point. This level of precision confirms that the model is reliable enough to be used as a diagnostic tool to identify students at risk.